

```

package main

import (
    "bytes"
    "fmt"
    "log"
    "os"
    "os/exec"
    "strings"
    "time"
)

type Config struct {
    ServiceName string
    Namespace    string
    ImageRepo    string
}

func main() {
    cfg := Config{
        ServiceName: envOrDefault("SERVICE_NAME", "my-service"),
        Namespace:   envOrDefault("NAMESPACE", "default"),
        ImageRepo:   envOrDefault("IMAGE_REPO",
"ghcr.io/my-org/my-service"),
    }

    version := buildVersionTag()
    image := fmt.Sprintf("%s:%s", cfg.ImageRepo, version)

    log.Printf("🔪 Deploying %s to namespace %s", cfg.ServiceName,
cfg.Namespace)
    log.Printf("📦 Image: %s", image)

    // 1) Build
    run("docker", "build", "-t", image, ".")

    // 2) Push
    run("docker", "push", image)

    // 3) Update Deployment
    run("kubectl", "-n", cfg.Namespace, "set", "image",
        "deployment/"+cfg.ServiceName,
        cfg.ServiceName+"="+image,
    )

    // 4) Wait for Rollout
    run("kubectl", "-n", cfg.Namespace, "rollout", "status",
"deployment/"+cfg.ServiceName)

    log.Println("✅ Deployment complete!")
}

func buildVersionTag() string {
    t := time.Now().UTC().Format("20060102-150405")

```

```

    sha := gitSHA()
    if sha == "" {
        return t
    }
    return fmt.Sprintf("%s-%s", t, sha)
}

func gitSHA() string {
    out, err := cmdOutput("git", "rev-parse", "--short", "HEAD")
    if err != nil {
        return ""
    }
    return strings.TrimSpace(out)
}

func envOrDefault(key, fallback string) string {
    v := os.Getenv(key)
    if v == "" {
        return fallback
    }
    return v
}

func run(name string, args ...string) {
    log.Printf("► %s %s", name, strings.Join(args, " "))
    cmd := exec.Command(name, args...)
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr
    if err := cmd.Run(); err != nil {
        log.Fatalf("✗ command failed: %v", err)
    }
}

func cmdOutput(name string, args ...string) (string, error) {
    cmd := exec.Command(name, args...)
    var buf bytes.Buffer
    cmd.Stdout = &buf
    cmd.Stderr = &buf
    err := cmd.Run()
    return buf.String(), err
}

```